

**Acta de Proyecto: EstudiaPro**

Plataforma web de organización académica

Nicole Tatiana Acosta Tellez

Ingeniería de Sistemas, Universidad Santo Tomás, Seccional Villavicencio

Gerencia de Software

7 de octubre de 2026

## **1. Nombre, propósito y justificación**

**EstudiaPro: Plataforma de organización académica.** La idea nace de una situación sencilla: un estudiante puede tener varias tareas, parciales, exámenes y cambios de horario, y terminar manejando todo en lugares distintos (cuadernos, chats, calendarios del celular). El proyecto reúne esa información en una sola aplicación web para que sea más fácil saber qué está pendiente, cuándo se debe entregar y cómo va el avance.

## **2. Problema y a quién afecta**

Muchos estudiantes universitarios tienen dificultades para organizar sus tareas, exámenes y horarios. Cuando la información está dispersa es más fácil olvidar una actividad, confundir una fecha o dejar para después algo importante, lo que termina generando estrés y afectando el rendimiento académico.

El problema se presenta principalmente en estudiantes de educación superior que manejan varias asignaturas y otras responsabilidades al mismo tiempo. La interlocutora principal del proyecto es Laura Valentina Rojas, estudiante de Ingeniería de Sistemas y usuaria potencial, quien aceptó participar en la validación de la solución.

## **3. Objetivos**

Desarrollar una aplicación web que permita registrar y consultar tareas, exámenes y horarios académicos durante el desarrollo del proyecto.

Implementar recordatorios que ayuden a tener presentes las actividades próximas.

Permitir que el estudiante marque el avance de sus actividades y consulte qué tiene pendiente.

Realizar una validación con la interlocutora principal antes de la entrega final y aplicar los ajustes que sean necesarios.

#### 4. Alcance

El alcance incluye el registro y la consulta de tareas académicas y de exámenes, la organización de horarios académicos, los recordatorios de actividades y fechas, y el seguimiento básico del estado de las actividades, en una aplicación web construida con React, Spring Boot y PostgreSQL.

La versión 1.0 excluye la conexión con sistemas de la universidad (matrícula o notas oficiales), una aplicación móvil nativa, las recomendaciones automáticas mediante inteligencia artificial, los módulos administrativos de la universidad y las integraciones externas que no sean necesarias para el funcionamiento básico del sistema.

#### 5. Supuestos y restricciones

*Supuestos:* se cuenta con un navegador web y conexión a Internet para usar la plataforma; el estudiante es quien registra y actualiza sus tareas, exámenes y demás actividades; y la interlocutora principal puede participar en la validación y dar su opinión sobre la solución.

*Restricciones:* la primera versión es únicamente web; no hay integración con los sistemas institucionales de matrícula o calificaciones; no se desarrollan funciones de inteligencia artificial; el proyecto lo desarrolla una sola persona; y el trabajo debe ajustarse al tiempo disponible dentro de la asignatura de Gerencia de Software.

*Criterio para pasar a «Terminado»:* la funcionalidad cumple su objetivo, no presenta errores conocidos que impidan su uso y cumple la definición de terminado de la sección 15.

## 6. Interesados

**Tabla 1**

*Interesados del proyecto*

| Interesado                      | Rol                      | Qué espera                                                                      | Poder | Interés |
|---------------------------------|--------------------------|---------------------------------------------------------------------------------|-------|---------|
| Laura Valentina Rojas           | Interlocutora principal  | Probar la idea y comentar si realmente le sirve para organizar sus actividades. | Medio | Alto    |
| Nicole Tatiana Acosta Tellez    | Responsable del proyecto | Analizar, desarrollar, probar y documentar la solución, y dejarla funcionando.  | Alto  | Alto    |
| Docente de Gerencia de Software | Evaluadora               | Ver evidencia del avance y del manejo del proyecto.                             | Alto  | Alto    |
| Estudiantes universitarios      | Usuarios potenciales     | Contar con una forma sencilla de organizar sus actividades académicas.          | Medio | Alto    |

## 7. Entregables e hitos

Las fechas exactas se alinean con el cronograma de la asignatura. La Tabla 2 registra, además, el estado de cada hito al 7 de octubre de 2026.

**Tabla 2**

*Entregables, hitos y estado*

| # | Entregable o hito                          | Resultado esperado                                                              | Fecha o estado                                                                  |
|---|--------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| 1 | Acta de constitución y tablero configurado | Alcance, objetivos, interesados, riesgos, roles y flujo de trabajo definidos.   | Cumplido                                                                        |
| 2 | Definición funcional                       | Lista priorizada de funcionalidades y flujo principal del usuario (sección 12). | Cumplido                                                                        |
| 3 | Primera versión funcional                  | Primer conjunto de funciones principales disponible para revisión.              | Cumplido                                                                        |
| 4 | Versión integrada                          | Tareas, exámenes, horarios, recordatorios y seguimiento trabajando en conjunto. | Cumplido (7 de octubre de 2026)                                                 |
| 5 | Pruebas y validación                       | Retroalimentación de la interlocutora registrada y ajustes aplicados.           | Semana 14: pruebas automáticas hechas; falta la validación con la interlocutora |
| 6 | Entrega final                              | Versión demostrable y documentación del proyecto.                               | Sesión 21: pendiente                                                            |

## 8. Estimación gruesa

La estimación se hace como rango porque el proyecto se desarrolla mientras se cursan otras materias. Se propone dedicar entre 10 y 16 horas por semana al proyecto, repartidas así: desarrollo, de 4 a 6 horas; análisis y diseño, de 2 a 4 horas; pruebas e integración, de 2 a 3 horas; y documentación y gestión, de 2 a 3 horas.

El reparto cambia según la etapa: en las primeras semanas pesa más el análisis y en las últimas, las pruebas y la validación con la interlocutora.

## 9. Matriz de riesgos

**Tabla 3**

*Riesgos principales y tratamiento*

| Riesgo                                                 | Prob. | Impacto | Respuesta planificada                                                                                                             |
|--------------------------------------------------------|-------|---------|-----------------------------------------------------------------------------------------------------------------------------------|
| Cambios en los requisitos después de la validación.    | Media | Alto    | Revisar el cambio antes de incluirlo y priorizar lo necesario para la versión 1.0.                                                |
| Retrasos por otras actividades académicas.             | Media | Alto    | Dividir el trabajo en tareas de menos de 4 horas y revisar el avance cada semana.                                                 |
| Errores al integrar frontend, backend y base de datos. | Media | Alto    | Probar cada parte por separado, integrar de forma gradual y mantener las pruebas automáticas del backend.                         |
| La interlocutora no tiene disponibilidad para validar. | Media | Medio   | Acordar la validación con anticipación, enviarle capturas y pasos, y dejar registradas sus observaciones.                         |
| El alcance crece durante el desarrollo.                | Alta  | Alto    | Mantener las exclusiones del acta y evaluar cualquier función nueva contra la definición de listo (sección 15).                   |
| Depender de una sola persona: no hay revisión cruzada. | Alta  | Medio   | Pruebas automáticas, lista de revisión en cada <i>pull request</i> y validación de la interfaz con la interlocutora (sección 14). |

*Nota.* La responsable de todos los riesgos es Nicole Tatiana Acosta Tellez.

## 10. Roles del proyecto

**Nicole Tatiana Acosta Tellez:** responsable única del proyecto. Organiza los requisitos, diseña y desarrolla el frontend y el backend, integra la base de datos PostgreSQL, ejecuta las

pruebas y documenta. Como no hay un segundo integrante, la revisión se apoya en las pruebas automáticas y en la lista de revisión de ESTANDARES.md.

**Laura Valentina Rojas (interlocutora principal):** prueba la aplicación como usuaria, comenta si le sirve para organizar sus actividades y actúa como retadora de las fichas de calidad (sección 11).

## 11. Requisitos y atributos de calidad

### 11.1 Clasificación de requisitos

Los requisitos de calidad del acta se organizan según las características del modelo de calidad de producto de ISO/IEC 25010 (International Organization for Standardization [ISO], 2011). Se agrega un requisito implícito de seguridad: cada estudiante solo puede ver y modificar su propia información.

**Tabla 4**

*Clasificación de requisitos según ISO/IEC 25010*

| ID    | Requisito                                                                                                        | Característica (subcaracterística)                               | Cómo se comprueba                                                               |
|-------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------|
| RC-01 | Las funciones principales se entienden sin una explicación extensa y los mensajes de error indican qué corregir. | Usabilidad (capacidad de aprendizaje, protección contra errores) | Prueba funcional con la interlocutora y revisión de los formularios (Figura 6). |
| RC-02 | Las pantallas principales no presentan esperas innecesarias.                                                     | Eficiencia de desempeño (comportamiento temporal)                | Ficha RC-02 (Tabla 6).                                                          |
| RC-03 | Se validan los campos obligatorios y no se guarda información incompleta o con fechas inválidas.                 | Adecuación funcional (corrección funcional)                      | Ficha RC-03 (Tabla 5) y pruebas automáticas.                                    |
| RC-04 | Lo que se registra, edita o cambia de estado se mantiene al volver a consultarlo.                                | Fiabilidad (madurez)                                             | Prueba crearEditarYCambiarEstado SeConserva.                                    |
| RC-05 | La aplicación funciona en Google Chrome y Microsoft Edge.                                                        | Portabilidad (adaptabilidad)                                     | Prueba de la versión final en los dos navegadores.                              |

| ID        | Requisito                                                                              | Característica (subcaracterística) | Cómo se comprueba                                                               |
|-----------|----------------------------------------------------------------------------------------|------------------------------------|---------------------------------------------------------------------------------|
| RC-06     | El código está separado por responsabilidades entre frontend, backend y base de datos. | Mantenibilidad (modularidad)       | Revisión de la estructura del proyecto (Figura 1 y Tabla 9).                    |
| Implícito | Cada estudiante ve y modifica solo su información.                                     | Seguridad (confidencialidad)       | Pruebas sinTokenNoSePuedeConsultarNada y unEstudianteNoVeNiTocaLasTareasDeOtro. |

## 11.2 Fichas de atributos de calidad

Se presentan dos fichas de siete campos para características distintas de ISO/IEC 25010. Cada ficha define la métrica, el instrumento, el umbral con el origen del número, el contexto, la regla de frontera y quién propone y quién reta la ficha (ISO, 2011).

### 11.2.1 Ficha 1 — Adecuación funcional (RC-03)

**Tabla 5**

*Ficha de atributo de calidad RC-03*

| Campo                               | Contenido                                                                                                                                                                                           |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Característica ISO/IEC 25010     | Adecuación funcional, subcaracterística corrección funcional.                                                                                                                                       |
| 2. Métrica                          | Número de registros inválidos (campos obligatorios vacíos, fechas pasadas, formatos incorrectos o clases que se cruzan) que el sistema acepta y guarda.                                             |
| 3. Instrumento                      | Pruebas de integración con JUnit 5 y MockMvc en backend/src/test (rechazaDatosIncompletosYFechasInvalidas, validaHorasYCruces, registroInvalidoIndicaCadaCampo) y prueba manual de los formularios. |
| 4. Umbral                           | 0 registros inválidos aceptados. Origen del número: RC-03 pide «evitar guardar información incompleta o con fechas inválidas»; cualquier registro inválido guardado incumple el requisito.          |
| 5. Contexto                         | Creación y edición de tareas, exámenes, horarios, asignaturas y registro de cuenta, tanto desde la API como desde los formularios web.                                                              |
| 6. Regla de frontera y comunicación | 0 cumple; 1 o más incumple. Ante un incumplimiento se abre un issue con el caso que entró, se agrega una prueba que lo reproduce y no se integra en main hasta que pase.                            |

| <b>Campo</b>             | <b>Contenido</b>                                                                                                                                                                                                                        |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7. Proponente y retadora | Proponente: Nicole Tatiana Acosta Tellez. Retadora: Laura Valentina Rojas, que intenta guardar datos incompletos o fechas inválidas durante la prueba funcional. La ficha se corrige o se justifica por escrito después de la revisión. |

### 11.2.2 Ficha 2 — Eficiencia de desempeño (RC-02)

**Tabla 6**

*Ficha de atributo de calidad RC-02*

| <b>Campo</b>                        | <b>Contenido</b>                                                                                                                                                                                                                     |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Característica ISO/IEC 25010     | Eficiencia de desempeño, subcaracterística comportamiento temporal.                                                                                                                                                                  |
| 2. Métrica                          | Tiempo de respuesta, en segundos, de las operaciones comunes: cargar el panel, listar tareas, crear una tarea y cambiar su estado.                                                                                                   |
| 3. Instrumento                      | Pestaña Red (Network) de las herramientas de desarrollo de Chrome, tomando 10 mediciones de cada operación contra el backend con la base de datos PostgreSQL.                                                                        |
| 4. Umbral                           | Promedio menor a 1,0 s por operación y ninguna medición de 2,0 s o más. Origen del número: el acta no fija una cifra; se toma 1,0 s porque es el límite en que el usuario mantiene la atención sin sentir la espera (Nielsen, 1993). |
| 5. Contexto                         | Uso normal de una cuenta con datos de un semestre (unas 50 tareas y 10 exámenes), en la demostración y en la sesión de validación.                                                                                                   |
| 6. Regla de frontera y comunicación | Un promedio de 1,0 s exactamente no cumple, porque el umbral dice «menor a 1,0 s». Si una operación incumple, se registra el caso, se revisa la consulta del servicio correspondiente y se vuelve a medir antes de la entrega final. |
| 7. Proponente y retadora            | Proponente: Nicole Tatiana Acosta Tellez. Retadora: Laura Valentina Rojas, que señala cualquier pantalla en la que perciba espera durante la prueba funcional.                                                                       |

## 11.3 Registro de revisión de las fichas

**Tabla 7**

*Proponente, retadora y estado de la revisión*

| <b>Ficha</b>                 | <b>Proponente</b>            | <b>Retadora</b>       | <b>Estado</b>                                                                                                           |
|------------------------------|------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------|
| RC-03 — Adecuación funcional | Nicole Tatiana Acosta Tellez | Laura Valentina Rojas | Pruebas automáticas en verde (0 registros inválidos aceptados). Falta la prueba manual con la retadora en la semana 14. |
| RC-02 — Eficiencia de        | Nicole Tatiana               | Laura Valentina       | Pendiente: medir con                                                                                                    |

## 12. Arquitectura y diseño del sistema

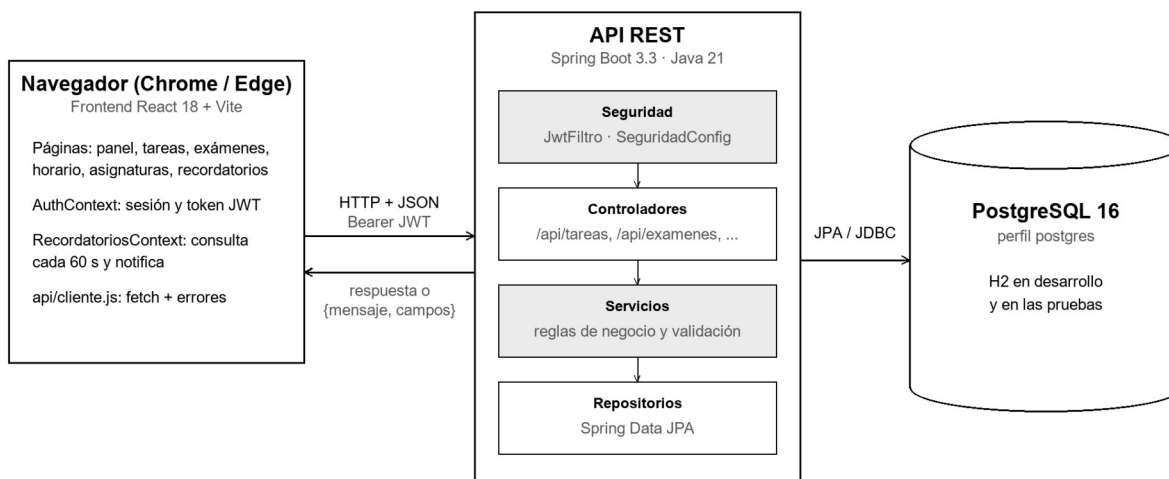
### 12.1 Arquitectura general

EstudiaPro sigue una arquitectura cliente-servidor en tres capas (Figura 1). El frontend es una aplicación de una sola página hecha con React (Meta Open Source, s. f.) que corre en el navegador. Se comunica con una API REST construida con Spring Boot (Spring, s. f.) mediante peticiones HTTP con cuerpo JSON. La API guarda los datos en PostgreSQL a través de Spring Data JPA. Esta separación responde a RC-06: cada capa se puede cambiar o probar por separado.

La sesión se maneja con un token JWT. El estudiante inicia sesión y recibe el token, y el frontend lo envía en cada petición. El backend no guarda sesiones: valida el token y toma de él el identificador del estudiante, que usa para filtrar todas las consultas.

**Figura 1**

*Arquitectura de EstudiaPro*



*Nota.* Elaboración propia. El backend usa H2 en archivo para trabajar sin instalar nada y PostgreSQL con el perfil `postgres`; las pruebas usan H2 en memoria.

## 12.2 Tecnologías

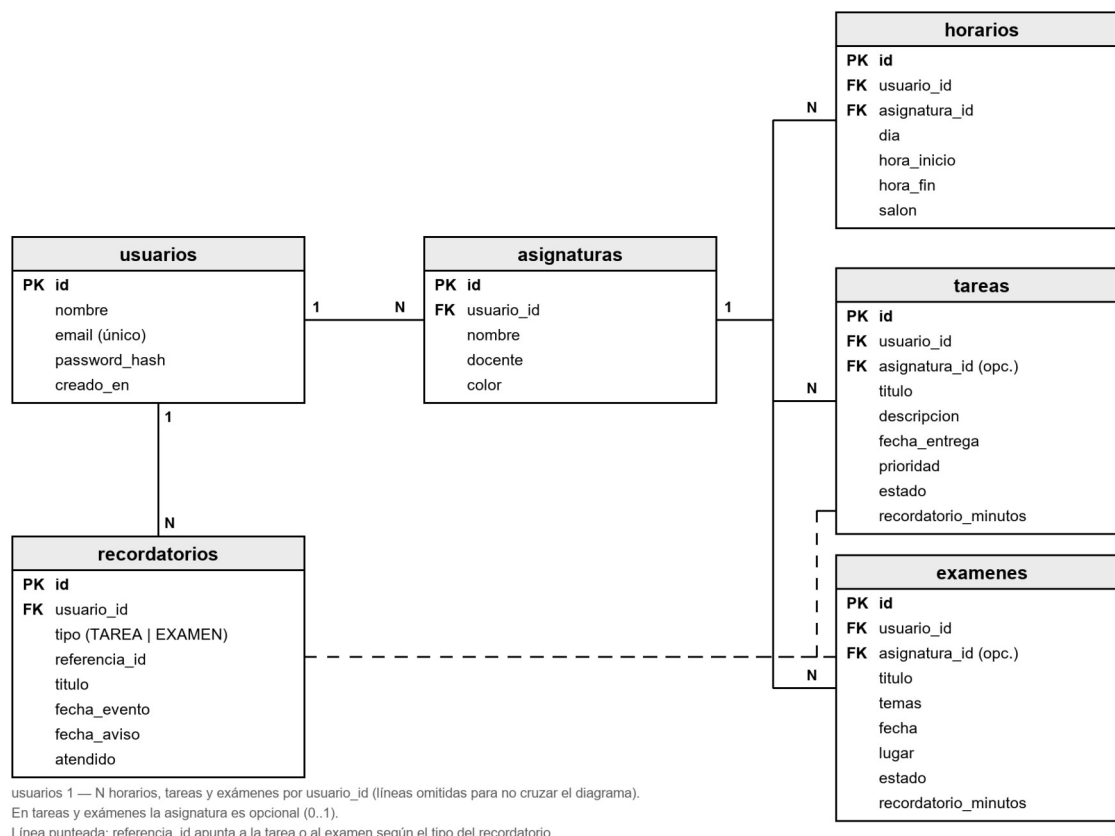
**Tabla 8**

*Tecnologías del proyecto*

| Capa     | Tecnología                  | Versión      | Uso                                                                        |
|----------|-----------------------------|--------------|----------------------------------------------------------------------------|
| Frontend | React + React Router        | 18.3 / 6.26  | Pantallas y navegación de la aplicación web.                               |
| Frontend | Vite                        | 5.4          | Servidor de desarrollo, proxy de /api y compilación.                       |
| Backend  | Java + Spring Boot          | 21 / 3.3.5   | API REST, validación y reglas de negocio.                                  |
| Backend  | Spring Security + JWT       | 6.3 / 0.12.6 | Autenticación sin estado con tokens JWT y contraseñas cifradas con BCrypt. |
| Datos    | Spring Data JPA (Hibernate) | 6.5          | Acceso a datos y generación del esquema.                                   |
| Datos    | PostgreSQL / H2             | 16 / 2.2     | Base de datos de la entrega / desarrollo y pruebas.                        |
| Pruebas  | JUnit 5 + MockMvc           | 5.10         | Pruebas de integración de la API.                                          |
| Gestión  | Git + GitHub                | —            | Control de versiones, ramas y pull requests.                               |

## 12.3 Modelo de datos

El modelo tiene seis tablas (Figura 2). Todas las tablas de información académica tienen la columna `usuario_id`, y por eso ningún estudiante puede ver ni modificar lo que registró otro. Una asignatura agrupa clases, tareas y exámenes. Las clases siempre pertenecen a una asignatura; en tareas y exámenes la asignatura es opcional. Los recordatorios guardan el tipo y el identificador de la actividad a la que apuntan, y la aplicación los crea, actualiza o borra sola al guardar la tarea o el examen.

**Figura 2***Modelo entidad-relación*

*Nota.* Elaboración propia a partir de las entidades JPA del paquete co.edu.usta.estudiapro. PK: llave primaria; FK: llave foránea.

## 12.4 Módulos del backend

Cada módulo del backend tiene su entidad, su repositorio, sus objetos de transferencia (DTO), su servicio y su controlador. Los controladores no contienen lógica: reciben la petición, la validan con Bean Validation y la pasan al servicio (Tabla 9).

**Tabla 9***Paquetes del backend y su responsabilidad*

| Paquete | Responsabilidad                                                                                   |
|---------|---------------------------------------------------------------------------------------------------|
| usuario | Registro, inicio de sesión y consulta de la cuenta. El correo se guarda en minúsculas y es único. |

| Paquete      | Responsabilidad                                                                                                           |
|--------------|---------------------------------------------------------------------------------------------------------------------------|
| seguridad    | Generación y validación del JWT, filtro de peticiones y configuración de CORS. Responde 401 en JSON cuando no hay sesión. |
| asignatura   | Materias del estudiante. No deja repetir nombres ni borrar una asignatura que tenga tareas, exámenes o clases.            |
| horario      | Clases de la semana. Valida que la hora de fin sea posterior a la de inicio y que no se crucen dos clases.                |
| tarea        | Tareas con prioridad (baja, media, alta) y estado (pendiente, en proceso, terminada). Calcula si están vencidas.          |
| examen       | Parciales y quices con fecha, lugar, temas y estado (pendiente o presentado).                                             |
| recordatorio | Avisos con la anticipación elegida. Desaparecen cuando la tarea se termina o el examen se presenta.                       |
| panel        | Resumen de la pantalla de inicio: conteos, porcentaje de avance, próximos 7 días y clases de hoy.                         |
| comun        | Formato único de errores {mensaje, campos} y conversión de excepciones en códigos 400, 401, 404 y 409.                    |

## 12.5 Interfaz de programación (API)

**Tabla 10**

### *Recursos de la API*

| Recurso       | Rutas (todas bajo /api)                                                                          | Reglas principales                                                                      |
|---------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Autenticación | POST /auth/registro<br>POST /auth/login<br>GET /auth/yo                                          | Contraseña de 6 a 72 caracteres; correo repetido → 409; credenciales incorrectas → 401. |
| Asignaturas   | GET, POST /asignaturas<br>PUT, DELETE /asignaturas/{id}                                          | Nombre repetido → 409; borrar con datos asociados → 409.                                |
| Horario       | GET, POST /horarios<br>PUT, DELETE /horarios/{id}                                                | Hora de fin anterior o igual → 400; cruce de clases → 409.                              |
| Tareas        | GET /tareas?estado=&asignaturaId=<br>POST, PUT, DELETE /tareas/{id}<br>PATCH /tareas/{id}/estado | Fecha de entrega en el pasado → 400; tarea de otro estudiante → 404.                    |
| Exámenes      | GET, POST /exámenes<br>PUT, DELETE /exámenes/{id}<br>PATCH /exámenes/{id}/estado                 | Fecha en el pasado → 400.                                                               |
| Recordatorios | GET /recordatorios<br>GET /recordatorios/pendientes<br>PATCH /recordatorios/{id}/atendido        | Pendientes: los que ya llegaron a su hora de aviso y no se han atendido.                |
| Panel         | GET /panel                                                                                       | Calcula el avance como tareas terminadas sobre el total.                                |

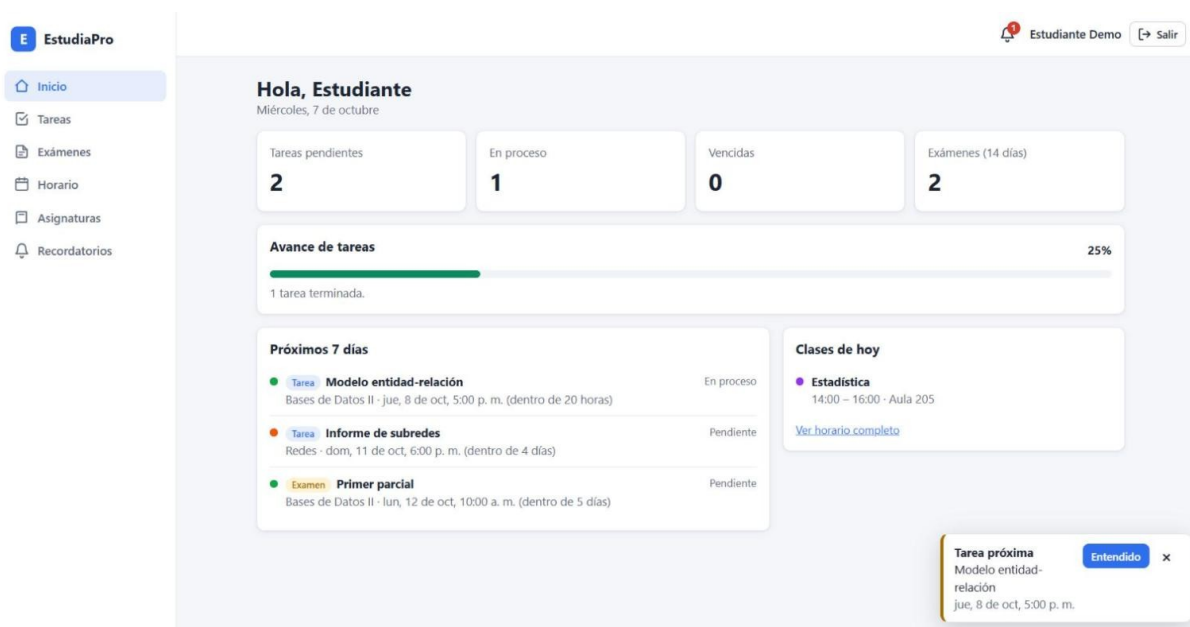
*Nota.* Todas las rutas, menos registro y login, exigen el encabezado Authorization: Bearer <token>. Sin token válido la respuesta es 401.

## 12.6 Interfaz de usuario

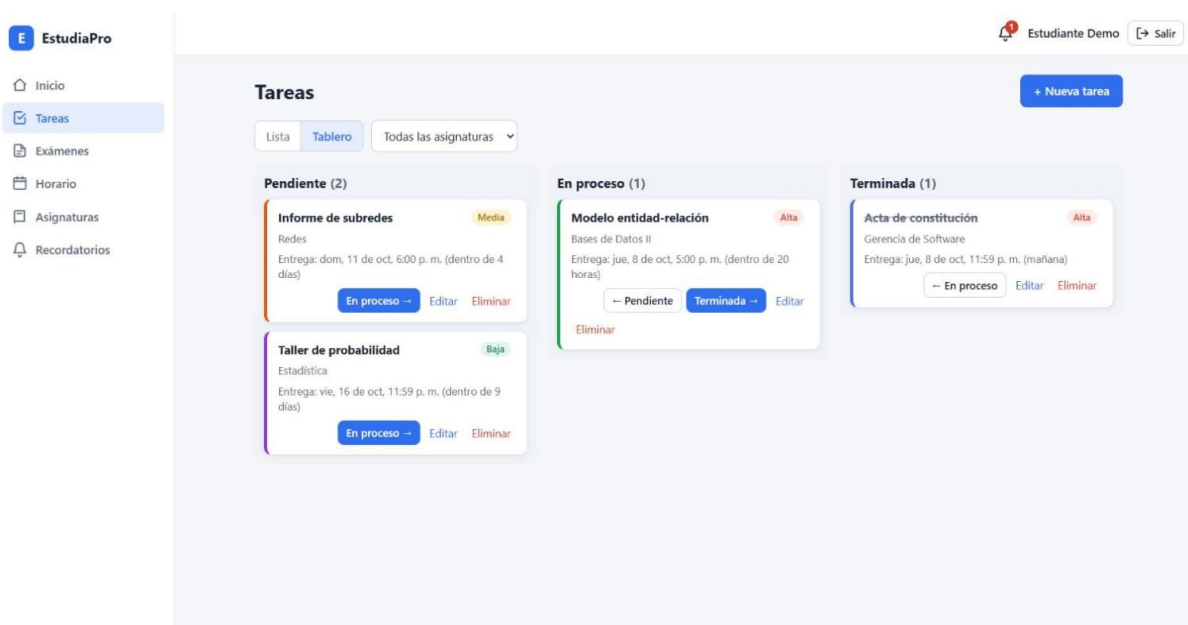
La aplicación tiene un menú lateral con seis pantallas: inicio, tareas, exámenes, horario, asignaturas y recordatorios. La pantalla de inicio resume lo pendiente y muestra el porcentaje de avance (Figura 3). Las tareas se pueden ver en lista o en un tablero con las columnas Pendiente, En proceso y Terminada, y se cambian de columna con un botón (Figura 4). El horario se ve como una cuadrícula semanal con un color por asignatura (Figura 5). Los formularios validan antes de enviar y muestran el error debajo de cada campo (Figura 6), como piden RC-01 y RC-03. Una campana en la parte superior cuenta los recordatorios pendientes: la aplicación los consulta cada minuto y, si el estudiante lo permite, también los muestra como notificación del navegador.

**Figura 3**

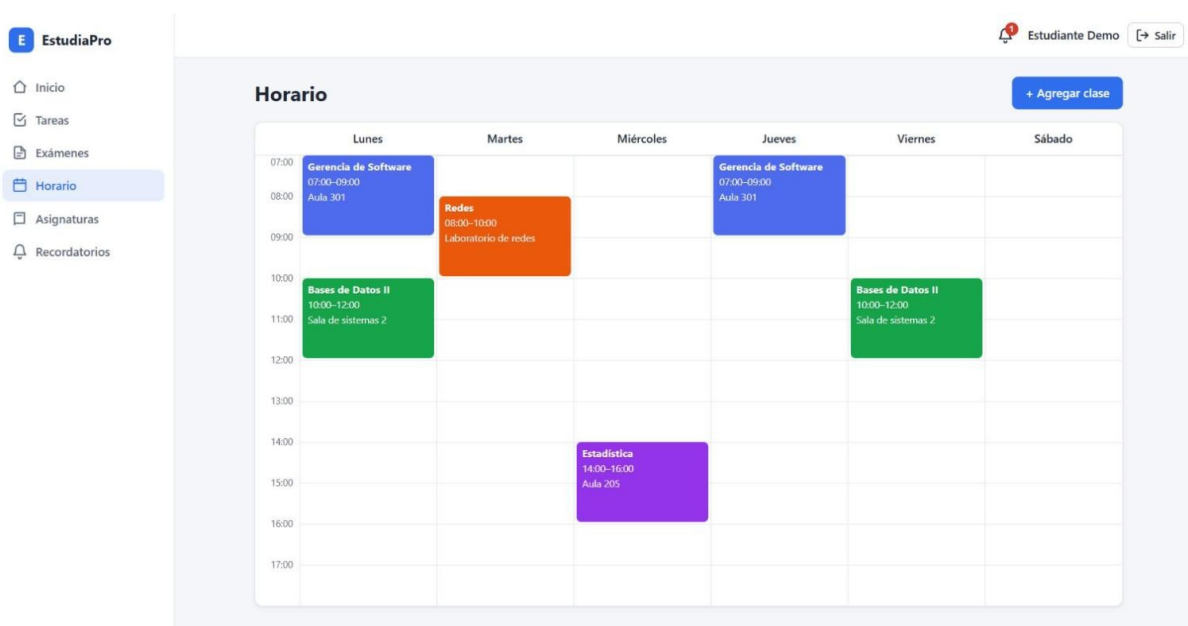
*Pantalla de inicio*



*Nota.* Captura de la aplicación en ejecución local con la cuenta de ejemplo, 7 de octubre de 2026. Abajo a la derecha aparece el aviso de un recordatorio pendiente.

**Figura 4***Tablero de tareas*

*Nota.* Captura del 7 de octubre de 2026. Cada tarjeta muestra la asignatura, la prioridad, la fecha de entrega y los botones para moverla de estado.

**Figura 5***Horario semanal*

*Nota.* Captura del 7 de octubre de 2026. Cada bloque muestra la asignatura, la franja horaria y el salón.

**Figura 6***Validación del formulario de tareas*

*Nota.* Captura del 7 de octubre de 2026 al intentar crear una tarea sin título ni fecha de entrega: el formulario no se envía y cada campo indica qué falta.

## 12.7 Reglas de negocio

1. Una tarea o un examen no se puede crear con fecha en el pasado. Al editar se puede conservar la fecha original aunque ya haya pasado, pero no moverla a otra fecha pasada.
2. Dos clases del mismo día no se pueden cruzar. Una clase puede empezar justo cuando termina otra.
3. Una asignatura con tareas, exámenes o clases no se borra; el sistema explica por qué y qué hacer.
4. El recordatorio se recalcula cada vez que cambia la fecha o la anticipación, y se borra cuando la tarea pasa a terminada o el examen a presentado.
5. Una tarea está vencida si su fecha ya pasó y no está terminada.

6. Ninguna consulta devuelve datos de otro estudiante: si el registro no le pertenece, la respuesta es 404.

### 13. Estándares del proyecto: historial de commits

El proyecto adopta Conventional Commits (Conventional Commits, s. f.) en su repositorio (<https://github.com/nicoleacosta2509-bot/EstudiaPro-USTA>). El backend está escrito en Java 21 con Spring Boot y el frontend en JavaScript con React. La Tabla 11 presenta los cinco commits de trabajo más recientes de la versión publicada el 7 de octubre de 2026 (commit `ada87c1`), sin contar los de integración entre ramas, con su hash corto, la autora, el mensaje original, el mensaje reescrito y el tipo asignado.

**Tabla 11**

*Cinco commits más recientes en formato Conventional Commits*

| Hash    | Autora        | Mensaje original                                                                    | Mensaje reescrito                                              | Tipo |
|---------|---------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------|------|
| 0265f79 | Nicole Acosta | docs(readme): enlazar ESTANDARES.md desde el README                                 | docs(readme): enlazar ESTANDARES.md desde el README            | docs |
| b21037d | Nicole Acosta | docs(estandares): agregar los estándares del proyecto                               | docs(estandares): agregar los estándares del proyecto          | docs |
| e4cbbb4 | Nicole Acosta | docs: arranque, API, pruebas y docker-compose de PostgreSQL                         | docs(readme): documentar arranque, API, pruebas y PostgreSQL   | docs |
| 4d64366 | Nicole Acosta | feat(frontend): interfaz React con panel, tareas, exámenes, horario y recordatorios | feat(frontend): construir la interfaz web con React y Vite     | feat |
| 6490b80 | Nicole Acosta | test(backend): pruebas de integración de RC-03, RC-04 y seguridad                   | test(backend): agregar pruebas de integración de RC-03 y RC-04 | test |

Los dos commits de documentación más recientes ya cumplían la convención. Los otros tres tenían problemas: el de documentación no tenía alcance, el del frontend pasaba de 72 caracteres y dos de ellos no empezaban con un verbo en infinitivo. La reescritura corrige esos puntos sin cambiar el tipo. Como ESTANDARES.md prohíbe reescribir el historial de main, la corrección se aplica de aquí en adelante.

## 14. Reglas de cierre de sprint

Los sprints duran una semana y se alinean con los hitos de la Tabla 2. Al terminar cada sprint, antes de empezar el siguiente, se verifican las reglas de la Tabla 12. Si una regla no se cumple, ese trabajo no se da por terminado y vuelve al backlog.

**Tabla 12**

*Reglas de cierre de sprint*

| Regla                | Criterio de cierre                                                                                                                                    | Responsable                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Terminado en main    | El trabajo está integrado en main, compila (mvn test y npm run build) y no incluye contraseñas ni llaves; los secretos viven en variables de entorno. | Nicole Tatiana Acosta Tellez                            |
| Revisión             | Cada pull request pasó la lista de revisión de ESTANDARES.md, revisada al día siguiente de terminar el cambio y no el mismo día.                      | Nicole Tatiana Acosta Tellez                            |
| Pruebas              | mvn test termina con 0 fallos. Si el sprint tocó formularios, se repite la prueba manual de RC-03; si tocó consultas, se vuelve a medir RC-02.        | Nicole Tatiana Acosta Tellez                            |
| Commits y versión    | Los commits siguen Conventional Commits. Al cerrar se integra dev en main y se crea la etiqueta de versión (v0.1.0 para el sprint 1).                 | Nicole Tatiana Acosta Tellez                            |
| Demo y retrospectiva | Se muestra el avance a la interlocutora y se anotan sus observaciones. Se registra qué salió bien, qué no y un compromiso para el siguiente sprint.   | Nicole Tatiana Acosta Tellez, con Laura Valentina Rojas |
| Pendientes           | Lo que no cumplió las reglas vuelve al backlog. Los cambios de alcance se deciden y se registran antes de empezar el siguiente sprint.                | Nicole Tatiana Acosta Tellez                            |

## 15. Definición de listo (DoR) y de terminado (DoD)

Para decidir cuándo una historia puede empezar y cuándo se da por terminada se usan dos acuerdos: la definición de listo (DoR, *Definition of Ready*) y la definición de terminado (DoD, *Definition of Done*), esta última propia del marco Scrum (Schwaber y Sutherland, 2020). La Tabla 13 muestra en qué se diferencian. Las Tablas 14 y 15 detallan los criterios de cada una y reproducen lo acordado en el archivo ESTANDARES.md del repositorio.

**Tabla 13***Diferencia entre la definición de listo y la de terminado*

|                  | <b>Definition of Ready (DoR)</b>         | <b>Definition of Done (DoD)</b>                                              |
|------------------|------------------------------------------|------------------------------------------------------------------------------|
| Cuándo se aplica | Antes de empezar la historia             | Antes de darla por terminada                                                 |
| Qué pregunta     | ¿Se entiende lo suficiente para empezar? | ¿Se hizo todo lo que había que hacer?                                        |
| Quién la revisa  | La responsable, al planear el sprint     | La responsable con la lista de revisión y, si hay interfaz, la interlocutora |
| Si no se cumple  | La historia no entra al sprint           | La historia no entra al incremento                                           |

**Tabla 14***Criterios de la definición de listo*

| # | <b>Condición</b>                                                                                                                           |
|---|--------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Existe como issue en GitHub, escrita como «como estudiante quiero [acción] para [beneficio]».                                              |
| 2 | Tiene criterios de aceptación escritos como casos: «entrada → resultado esperado», con los códigos de respuesta cuando es una API.         |
| 3 | Se puede terminar en menos de 4 horas de trabajo. Si no, se parte.                                                                         |
| 4 | Está dentro del alcance del acta. Si es una función nueva, ya se evaluó contra las exclusiones.                                            |
| 5 | Si toca backend y frontend a la vez, la ruta, el método, el cuerpo y los códigos de respuesta están escritos en el issue antes de empezar. |
| 6 | Si cambia algo que la interlocutora va a usar, se sabe cuándo se le va a mostrar.                                                          |

**Tabla 15***Criterios de la definición de terminado*

| # | <b>Condición</b>                                                                                  | <b>Cómo se comprueba</b>                                                            |
|---|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1 | Las pruebas del backend pasan.                                                                    | cd backend && mvn test termina sin fallos.                                          |
| 2 | Cada criterio de aceptación que toca la API tiene al menos una prueba automática.                 | En backend/src/test hay una prueba cuyo nombre describe ese criterio, y pasa.       |
| 3 | El frontend compila.                                                                              | cd frontend && npm run build termina con código 0.                                  |
| 4 | Si el cambio es de interfaz, el pull request trae una captura y los pasos para verlo.             | El pull request incluye la imagen y los pasos con npm run dev.                      |
| 5 | Los formularios validan antes de enviar y muestran el error junto al campo.                       | Se envía el formulario vacío y con una fecha pasada; cada campo muestra su mensaje. |
| 6 | Si cambia un endpoint, una variable de entorno o un comando, el README cambia en el mismo cambio. | El diff del pull request incluye README.md, backend/README.md o frontend/README.md. |

| # | Condición                                                         | Cómo se comprueba                                                    |
|---|-------------------------------------------------------------------|----------------------------------------------------------------------|
| 7 | El pull request pasó la lista de revisión.                        | La lista aparece marcada en la descripción del pull request.         |
| 8 | Si la historia cambia algo que usa la interlocutora, ella lo vio. | Su observación queda anotada en el issue o en el acta de validación. |

La Tabla 16 presenta ejemplos de puntos de la DoD del proyecto. Cada punto describe algo que se puede observar y nombra la evidencia que lo comprueba. Las pruebas citadas existen en backend/src/test y pasaron el 7 de octubre de 2026.

**Tabla 16**

*Ejemplos de puntos de definición de terminado del proyecto*

| #  | Punto de DoD                                                                                                                | Evidencia                                                    |
|----|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 1  | El estudiante se registra, inicia sesión y consulta su cuenta; el correo no distingue mayúsculas.                           | La prueba registroLoginYConsultaDelUsuario pasa en mvn test. |
| 2  | El registro rechaza con 400 un nombre vacío, un correo mal escrito y una contraseña corta, e indica el error de cada campo. | La prueba registroInvalidoIndicaCadaCampo pasa.              |
| 3  | La API responde 401 si no hay token o si el token es falso.                                                                 | La prueba sinTokenNoSePuedeConsultarNada pasa.               |
| 4  | Una tarea creada, editada y cambiada de estado conserva sus datos al volver a consultarla (RC-04).                          | La prueba crearEditarYCambiarEstadoSeConserva pasa.          |
| 5  | La API rechaza con 400 una tarea sin título, sin fecha, con fecha pasada o con fecha mal escrita, y no guarda nada (RC-03). | La prueba rechazaDatosIncompletosYFechasInvalidas pasa.      |
| 6  | Un estudiante no ve ni puede borrar las tareas de otro.                                                                     | La prueba unEstudianteNoVeNiTocaLasTareasDeOtro pasa.        |
| 7  | El recordatorio de una tarea aparece cuando llega su hora de aviso y desaparece al terminarla.                              | La prueba elRecordatorioSigueElEstadoDeLaTarea pasa.         |
| 8  | El panel calcula el avance como tareas terminadas sobre el total.                                                           | La prueba panelResumeElAvance pasa.                          |
| 9  | El horario rechaza con 400 una clase que termina antes de empezar y con 409 una que se cruza con otra.                      | La prueba validaHorasYCruces pasa.                           |
| 10 | No se puede borrar una asignatura que tiene clases ni crear dos con el mismo nombre.                                        | La prueba noBorraUnaAsignaturaConClases pasa.                |

## 16. Estrategia de ramas

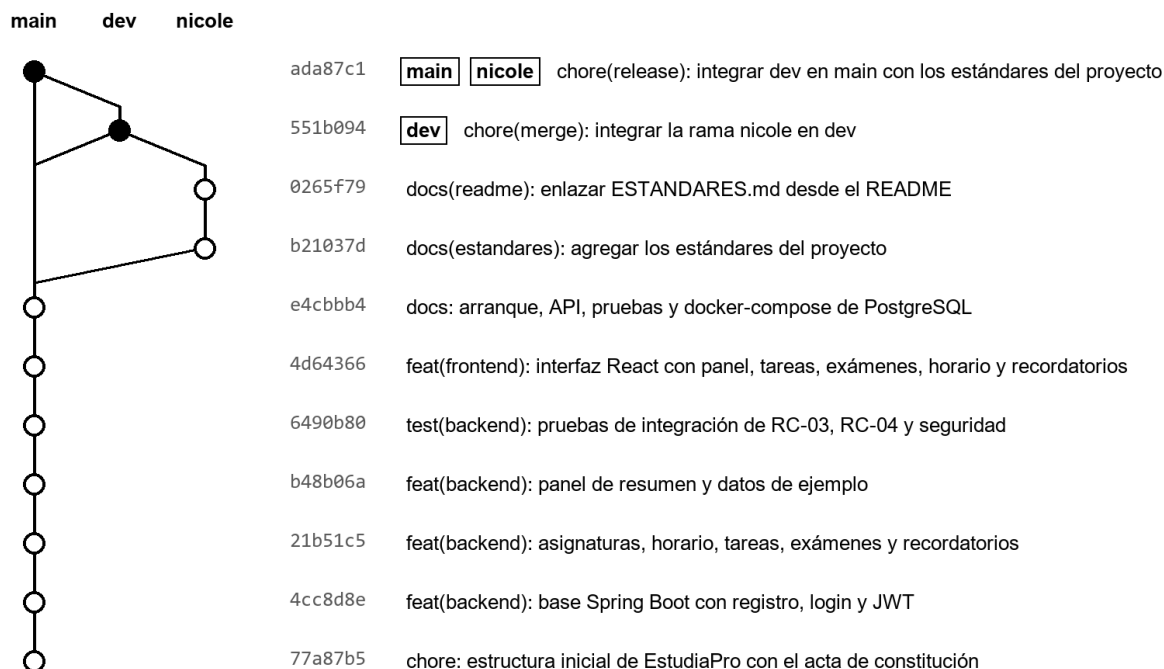
El repositorio tiene tres ramas: main, dev y la rama de trabajo nicole. Los commits se hacen en nicole; nicole se integra en dev y dev se integra en main al cerrar el sprint, como muestra la Figura 7. No se trabaja directamente en main ni en dev: los cambios llegan a dev mediante una solicitud de integración (pull request) que debe pasar la lista de revisión.

**Tabla 17**

*Ramas del repositorio y su función*

| Rama   | Propósito                                                   | Responsable                  | Se integra en                        |
|--------|-------------------------------------------------------------|------------------------------|--------------------------------------|
| main   | Versión estable y entregable del proyecto.                  | Nicole Tatiana Acosta Tellez | Recibe desde dev al cerrar el sprint |
| dev    | Integración de lo que ya pasó la lista de revisión.         | Nicole Tatiana Acosta Tellez | main, al cerrar el sprint            |
| nicole | Trabajo diario: backend, frontend, pruebas y documentación. | Nicole Tatiana Acosta Tellez | dev, mediante pull request           |

*Flujo de integración:* (1) se trabaja y se hacen commits en nicole; (2) se abre un pull request hacia dev y se revisa con la lista de ESTANDARES.md según la definición de terminado (Tabla 15); (3) al cerrar el sprint, dev se integra en main sin conflictos y se crea la etiqueta de versión.

**Figura 7***Historial de ramas del repositorio*

*Nota.* Elaboración propia a partir de `git log --graph` del repositorio [github.com/nicoleacosta2509-bot/EstudiaPro-USTA](https://github.com/nicoleacosta2509-bot/EstudiaPro-USTA), 7 de octubre de 2026 (11 commits). Los círculos rellenos son commits de integración: nicole entra en dev (`chore(merge)`) y dev entra en main (`chore(release)`).

## 17. Pruebas y resultados

El 7 de octubre de 2026 se ejecutó `mvn test` en el backend: 11 pruebas de integración, 0 fallos y 0 errores. Las pruebas levantan la aplicación completa con una base H2 en memoria y hacen peticiones reales a la API, de modo que comprueban la validación, la seguridad y la persistencia juntas. El frontend compiló sin errores con `npm run build`. La Tabla 18 relaciona cada clase de prueba con los requisitos que cubre.

**Tabla 18***Pruebas automáticas y requisitos que cubren*

| Clase de prueba        | Pruebas | Requisitos                                        | Resultado    |
|------------------------|---------|---------------------------------------------------|--------------|
| AuthIntegracionTest    | 4       | RC-01, RC-03, seguridad                           | 4 de 4 pasan |
| TareaIntegracionTest   | 5       | RC-03, RC-04, seguridad, recordatorios, panel     | 5 de 5 pasan |
| HorarioIntegracionTest | 2       | RC-03 (cruces y horas), integridad de asignaturas | 2 de 2 pasan |

Quedan pendientes y se registran para la semana 14: la prueba funcional con la interlocutora (RC-01 y la parte manual de RC-03), la medición de tiempos de RC-02 con PostgreSQL y la prueba de la versión final en Microsoft Edge (RC-05). La aplicación ya se probó en Google Chrome, de donde salen las Figuras 3 a 6.

### 18. Tablero configurado del proyecto

El tablero tiene cuatro estados para seguir el trabajo sin complicarlo: Pendiente, En proceso, En revisión y Terminado. Cada tarea pasa de izquierda a derecha a medida que avanza. La Tabla 19 reproduce el tablero tal como quedó configurado en el acta de constitución.

**Tabla 19***Tablero configurado en el acta de constitución*

| Pendiente               | En proceso                          | En revisión                   | Terminado            |
|-------------------------|-------------------------------------|-------------------------------|----------------------|
| Diseño de interfaz      | Definir funcionalidades principales | Validar alcance y prioridades | Ficha de proyecto    |
| Modelo de base de datos | Preparar estructura inicial         | Revisar con la interlocutora  | Acta de constitución |
| Integración de módulos  | Implementar tareas y exámenes       | Probar flujo completo         | —                    |
| Pruebas finales         | Implementar recordatorios           | Revisión de pendientes        | —                    |

Reglas de uso: cada tarea tiene una responsable antes de pasar a En proceso; pasa a En revisión cuando está lista para comprobarse; solo pasa a Terminado cuando cumple la definición de terminado (Tabla 15); y el tablero se revisa en el seguimiento semanal.

Al 7 de octubre de 2026, el diseño de interfaz, el modelo de base de datos, la estructura inicial, la integración de módulos, las tareas y exámenes y los recordatorios ya están contruidos y con pruebas. Quedan en revisión la validación con la interlocutora, el flujo completo en Edge y las pruebas finales.

## 19. Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). <https://doi.org/10.1037/0000165-000>
- Conventional Commits. (s. f.). *Conventional Commits 1.0.0*.  
<https://www.conventionalcommits.org/es/v1.0.0/>
- International Organization for Standardization. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*.
- Meta Open Source. (s. f.). *React*. <https://react.dev/>
- Nielsen, J. (1993). *Usability engineering*. Academic Press.
- Schwaber, K., y Sutherland, J. (2020). *La guía de Scrum*.  
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf>
- Spring. (s. f.). *Spring Boot reference documentation*. <https://docs.spring.io/spring-boot/>